detonationFoam for OpenFOAM Foundation 14

Theory, Usage, OF8 Migration, Automatic Unrefinement, Soret Compatibility, and Qualification

Version 1.1.0 - September 2026

Release scope. This manual documents OpenFOAM Foundation 14 `detonationFoam` v1.1.0. The solver is delivered as the `foamRun` module `detonationFluid`. The reusable two-dimensional AMR component is `planarRefiner`, an `fvMeshTopoChanger` in `libplanarFvMeshTopoChangers.so`. Version 1.1.0 adds opt-in reversible automatic unrefinement and an explicit `publishedOF8` H/H2 Soret compatibility mode. Both new behaviors default to off.

Table of Contents

1. Introduction

The OpenFOAM 14 port preserves the density-based reacting detonation formulation and the legacy detonation flux-family choices while adopting the modular OpenFOAM 14 `foamRun` architecture. The primary solver library is:

```
libdetonationFluidSolver.so
```

and cases select it with:

```
solver detonationFluid;
```

in `system/controlDict`, then execute with:

```
foamRun
```

The port deliberately uses OpenFOAM 14 native capability where it is equivalent and maintains targeted compatibility code only where the OpenFOAM 8 behavior was not reproduced by a stock OpenFOAM 14 model. The largest compatibility layer is the exact legacy `NS_mixtureAverage` transport/property path, implemented through the standard OpenFOAM 14 multicomponent thermophysical-transport interfaces rather than by restoring a separate legacy solver branch.

The release also contains a reusable dynamic 2-D/axisymmetric refiner:

```
libplanarFvMeshTopoChangers.so
```

with runtime type:

```
planarRefiner
```

This AMR library is intentionally independent of detonationFoam.

Version 1.1.0 adds two qualified opt-in extensions: automatic reversible coarsening in `planarRefiner`, and `legacyThermalDiffusionMode publishedOF8` in `legacyMixtureAverageFourier`. The Soret mode reproduces the source behavior published in the supplied OF8 implementation, including its H2-to-H assignment, and is therefore a reproducibility mode rather than a corrected-Soret claim.

2. Package architecture

2.1 Solver module

`applications/modules/detonationFluid/` contains the `foamRun` solver module. Its architecture follows the OpenFOAM 14 density-based shock-fluid pattern while adding reacting multicomponent chemistry, the restored detonation flux families, and optional viscous/multicomponent transport.

The main field set is:

- pressure p;

- density ρ;
- velocity U;
- mass flux ϕ;
- specific internal energy e;
- species mass fractions Y_i;
- chemical heat-release rate $\dot{Q}$.

`fvModels` and `fvConstraints` remain available through normal OpenFOAM 14 interfaces in the density, momentum, species, and energy equations.

2.2 Legacy transport compatibility library

`src/detonationLegacyThermophysicalTransportModels/legacyMixtureAverageFourier/` builds:

`libdetonationLegacyThermophysicalTransportModels.so`

This library provides:

- `legacyMixtureAverageFourier`;
- `legacyBinaryDiffusionCoefficient`;
- the runtime registration needed for the converted `logPolynomial` species transport data with `coefficientWilkeMulticomponentMixture`.

It is used only when exact legacy transport/property behavior is desired.

2.3 Reusable 2-D/wedge AMR library

`src/planarFvMeshTopoChangers/` builds:

`libplanarFvMeshTopoChangers.so`

The runtime type is `planarRefiner`. The library owns topology refinement for slab/`empty` and wedge/axisymmetric meshes and contains no dependency on `detonationFluid`.

> **Terminology note:** the 2-D AMR component is sometimes informally called the "2-D AMR fvModel". In OpenFOAM 14 it is correctly implemented as an **`fvMeshTopoChanger`**, because topology change is a mesh operation. `fvModel` remains the equation/source-term abstraction.

3. Governing theory

3.1 Continuity: $\partial_t \rho + \nabla \cdot (\rho U) = S_\rho$

The density equation advanced by `correctDensity.C` is

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho U) = S_\rho .$$

where S_ρ represents any source supplied through `fvModels().source(rho)`. The face mass flux is constructed by the selected shock-capturing flux backend rather than by a pressure-based flux correction.

3.2 Momentum: $\partial_t (\rho U) + \nabla \cdot F_{\rho U} = \nabla \cdot \tau + S_U$

The conservative momentum equation is represented as

$$\frac{\partial (\rho U)}{\partial t} + \nabla \cdot F_{\rho U} = \nabla \cdot \tau + S_U .$$

For `Euler`, the viscous stress term is disabled. For the viscous path, the stress operator comes from the OpenFOAM 14 compressible momentum-transport model through `momentumTransport_->divDevTau(U)`.

The numerical momentum flux contains both the transported momentum and the pressure contribution. HLLCP additionally supplies explicit conservative pressure-difference corrections.

3.3 Species: $\partial_t(\rho Y_i)+\nabla\cdot(\phi Y_i)=\dot{\omega}_i-\nabla\cdot j_i+S_i$

For each solved species,

$$\frac{\partial(\rho Y_i)}{\partial t}+\nabla\cdot(\rho U Y_i)=\dot{\omega}_i-\nabla\cdot j_i+S_i.$$

where $\dot{\omega}_i$ is the finite-rate chemistry source, j_i is the diffusive mass flux for viscous modes, and S_i is an optional `fvModel` source.

After the species equations are solved, OpenFOAM's multicomponent thermo normalizes the mass fractions:

$$\sum_i Y_i=1.$$

The qualification suite explicitly checks this closure after transport, AMR mapping, MPI redistribution, and restart.

3.4 Total-energy form: $e+K$ with chemical heat release

The solver advances sensible internal energy e together with the kinetic-energy contribution $K=|U|^2/2$. In continuous notation the implemented balance corresponds to

$$\frac{\partial(\rho e)}{\partial t}+\nabla\cdot F_E+\frac{\partial(\rho K)}{\partial t}=\dot{Q}+S_e+\nabla\cdot q_{diff}+\nabla\cdot(\tau\cdot U).$$

with the diffusive terms omitted for `Euler`. The convective energy flux contains reconstructed internal energy, kinetic energy, and the pressure-work flux supplied by the selected Riemann/central-upwind backend. Mesh-motion pressure work is included for moving meshes.

The chemistry model supplies the chemical heat-release rate

$$\dot{Q}=\dot{Q}_{chem},$$

which is obtained from the OpenFOAM chemistry interface `reaction_->Qdot()`.

3.5 Thermodynamics: $p=\rho/\psi$

After the conservative update, pressure is recovered from the OpenFOAM thermodynamic compressibility ψ:

$$p=\frac{\rho}{\psi}.$$

The converted reference mechanism uses a perfect-gas equation of state with JANAF thermochemistry and sensible internal energy. The exact legacy transport case uses:

```
mixture         coefficientWilkeMulticomponentMixture;
transport       logPolynomial;
thermo          janaf;
energy          sensibleInternalEnergy;
equationOfState perfectGas;
```

3.6 Acoustic time-step control: Co_a

The density-based time-step limit is based on an acoustic face signal speed. In compact form,

$$Co_a = \frac{1}{2} \Delta t \max_c \left[\frac{\sum_{f \in c} a_{max,f} |S_f)}{V_c} \right).$$

The local-time-stepping form uses the same signal-speed measure to set the reciprocal time scale. Standard cases use `adjustTimeStep yes` and `maxCo` in `controlDict`/PIMPLE settings.

4. Numerical flux families

4.1 Runtime selection: `fluxScheme`

The flux backend is selected in `system/fvSchemes`:

```
fluxScheme HLLCP;
```

The release supports seven backends:

Flux	Release status	Role
`Kurganov`	qualified	central-upwind baseline
`Tadmor`	qualified	central scheme
`HLL`	restored and qualified	two-wave HLL Riemann flux
`HLLC`	restored and qualified	HLL with contact-wave restoration
`HLLCP`	restored and qualified	pressure-corrected HLLC-family detonation flux
`AUSM+`	restored and qualified	AUSM-family convective/pressure splitting
`AUSM+up`	restored and qualified	AUSM+ with pressure/velocity dissipation controls

All backends feed a unified conservative OpenFOAM 14 face-flux path. The HLLC/HLLCP implementations retain contact/star-state logic without creating a separate solver branch.

4.2 Reconstruction

The reference cases use OpenFOAM 14 `shockFluid`-style reconstruction keys, for example:

```
reconstruct(rho)    Minmod;
reconstruct(U)      MinmodV;
reconstruct(T)      Minmod;
```

The species and thermodynamic fields use the common face directions generated by the selected flux backend.

5. Chemistry and transport

5.1 OpenFOAM 14 finite-rate chemistry

The legacy DLBFoam chemistry/load-balancing implementation is not carried forward as a separate chemistry engine. OpenFOAM 14 standard chemistry is used instead:

```
type       standard;
chemistry  on;
cpuLoad    true;
```

`cpuLoad true` registers chemistry cost information that can be consumed by the native OpenFOAM 14 `loadBalancer` in parallel cases.

5.2 `Euler`

`solverType Euler;` disables viscous momentum, species diffusion, and conductive/diffusive energy transport while retaining finite-rate chemistry and the selected shock-capturing flux.

5.3 `NS_Sutherland`

`solverType NS_Sutherland;` activates the viscous transport path. For a conventional Sutherland case, native OpenFOAM 14 momentum/thermophysical transport is used. The fixed-time OF8-to-OF14 comparison for this path passed with identical shock location and small profile differences.

5.4 Exact legacy `NS_mixtureAverage` behavior

The OpenFOAM 8 `NS_mixtureAverage` equation branch is **not** restored as a separate solver type. Instead, its missing physics is implemented through normal OpenFOAM 14 runtime interfaces.

For species i, the legacy mixture-averaged diffusion coefficient is

$$D_i = \frac{1 - Y_i}{\sum_{j \neq i} \frac{X_j}{D_{ij}}}.$$

The release evaluates the raw positive diffusion vector

$$F_i = \rho D_i \nabla Y_i + Y_i \rho D_i \frac{\nabla W_{mix}}{W_{mix}}.$$

and exposes the corrected physical species flux

$$j_i = -F_i + Y_i \sum_k F_k.$$

which enforces

$$\sum_i j_i = 0$$

up to numerical roundoff.

The exact OF8 binary-diffusion correlation is

$$D_{ij}(p, T) = 10^{-4} \frac{\exp\left[D_1 + \ln(T)\left(D_2 + \ln(T)\left(D_3 + \ln(T) D_4\right)\right)\right)}{p/101325},$$

with p in Pa, T in K, and D_{ij} in $m^2 s^{-1}$.

Species viscosity and conductivity use the converted `muLogCoeffs<8>` and `kappaLogCoeffs<8>` through native OpenFOAM 14 `logPolynomialTransport`. Mixture viscosity uses native `coefficientWilkeMulticomponentMixture`, which was verified algebraically identical to the legacy Wilke rule.

Legacy mixture conductivity is retained as the arithmetic/harmonic average

$$\lambda_A = \sum_i X_i \lambda_i, \lambda_B = \sum_i \frac{X_i}{\lambda_i},$$

$$\lambda_{mix} = \frac{1}{2}\left(\lambda_A + \frac{1}{\lambda_B}\right).$$

A case activates this path with `legacyMixtureAverageFourier` in `constant/thermophysicalTransport` and loads `libdetonationLegacyThermophysicalTransportModels.so` from `controlDict`.

5.5 Soret/thermal diffusion

The default remains:

```
legacyThermalDiffusionMode off;
```

Version 1.1.0 adds the explicit compatibility selector:

```
legacyThermalDiffusionMode publishedOF8;
```

This mode deliberately reproduces the **published OF8 source behavior** rather than silently correcting it. Let

$$P_{ij}(\theta) = a_1 + \theta\left[a_2 + \theta\left(a_3 + \theta a_4\right)\right), \theta = \frac{T}{1K},$$

where the four coefficients are the legacy `ThermDiff_1` through `ThermDiff_4` values for a light-species pair. The published OF8 assignment reproduced by this release is

$$R_H = \sum_{j \neq H} X_H X_j P_{Hj}(\theta) + \sum_{j \neq H_2} X_{H_2} X_j P_{H_2 j}(\theta),$$

while

$$R_{H_2} = 0.$$

Thus both the H and H2 polynomial sums accumulate into the H thermal-diffusion ratio, and the direct H2 thermal-diffusion ratio remains zero. This is the anomalous assignment present in the supplied OF8 source and is preserved only for reproducibility.

For light species i, the raw thermal-diffusion contribution has the form

$$F_{i,T} = -\rho D_i \frac{R_i Y_i}{T X_i} \nabla T,$$

with the normal OpenFOAM phase/transport weighting applied during face interpolation. The thermal contribution is added to the same raw mixture-diffusion flux used by the legacy compatibility model, and the final physical species flux remains

$$j_i = -F_i + Y_i \sum_k F_k,$$

so that

$$\sum_i j_i = 0$$

to numerical roundoff. Even though the direct H2 Soret term is zero in `publishedOF8` mode, the H thermal term contributes to the common zero-net-mass correction and can therefore influence the corrected fluxes of other species.

The required coefficients can be supplied in either of two layouts:

1. an inline `thermalDiffusionCoeffs` dictionary in the `legacyMixtureAverageFourier` coefficients; or
2. the original OF8 `constant/thermoDiff` dictionary.

Pair names may be `H-X`/`X-H` and `H2-X`/`X-H2`, using the keys `ThermDiff_1` through `ThermDiff_4`. Explicit pair entries are required. The v1.1.0 compatibility layer intentionally does **not** infer missing data through the old `trandat/groupSpecies` coefficient-sharing shortcut.

The supplied OF8 source archive contained the reader and source implementation but did **not** contain a physical `constant/thermoDiff` coefficient dataset. No physical coefficient values are invented in this release. The packaged Soret qualification and laptop smoke therefore use synthetic diagnostic coefficients only to exercise and verify the code path.

The direct OpenFOAM 14 flux qualification passed the published H/H2 identity and zero-net-mass closure at relative residuals of approximately 1.6×10^{-16} and 1.3×10^{-16}, respectively, with an exactly zero Soret-off control.

6. Dynamic mesh, AMR, and load balancing

6.1 Native 3-D AMR

For genuine 3-D cases, OpenFOAM 14's native `fvMeshTopoChangers::refiner` is the preferred path. The migration qualified serial refinement, fixed-mesh versus localized-AMR behavior, MPI redistribution, and restart.

6.2 Reusable 2-D/axisymmetric AMR: `planarRefiner`

The solver-independent `planarRefiner` supports true 2-D meshes with:

- slab/`empty` geometry;
- wedge/axisymmetric geometry;
- localized scalar-field band selection;
- refinement-only MPI distribution callbacks and refined/distributed restart;
- optional reversible automatic unrefinement with a two-sided hysteresis band;
- per-cell binary cut-depth tracking for repeated refine/unrefine cycles;
- fixed-decomposition MPI operation in reversible mode;
- operation with stock `incompressibleFluid`, demonstrating portability beyond detonationFoam.

For one planar refinement level, each selected parent is split in the two active geometric directions into four children, giving

$$N_{new}=N_{old}+3N_{selected}.$$

A representative reversible dictionary is:

```
FoamFile
{
    format   ascii;
    class    dictionary;
    object   dynamicMeshDict;
}

topoChanger
{
    type                        planarRefiner;
    libs                        ("libplanarFvMeshTopoChangers.so");
    geometry                    slab;      // auto | slab | wedge
    refineInterval              1;
    field                       T;
    lowerRefineLevel            800;
    upperRefineLevel            4900;
    maxCells                    200000;
    maxRefinementIterations     1;         // refinement-only mode cap
```

```
    automaticUnrefinement        true;
    unrefineInterval             1;
    lowerUnrefineLevel           700;       // <= lowerRefineLevel
    upperUnrefineLevel           5000;      // >= upperRefineLevel
    maxRefinementLevel           1;
    maxUnrefinementPassesPerUpdate 2;
}
```

`automaticUnrefinement` defaults to `false`, so v1.0.0 behavior is preserved. When enabled, a reversible split is eligible for coarsening only when both sibling cells have moved to the **same outside side** of the hysteresis band:

$$\phi_o, \phi_n < L_u \text{ or } \phi_o, \phi_n > U_u,$$

where $L_u \le L_r$ and $U_u \ge U_r$. A split that still straddles the shock/reaction indicator is retained.

One planar level consists of two binary directional cuts, so the default `maxUnrefinementPassesPerUpdate 2` can collapse four planar children back to their parent when both undo levels are eligible. `maxRefinementLevel` is enforced through per-cell binary cut depth, which allows repeated refine $\rightarrow$ unrefine $\rightarrow$ refine cycles without exhausting the old global event counter.

The v1.1.0 reversible runtime gate passed uniform slab, localized slab, and annular wedge cycles, including `1200 -> 4800 -> 1200 -> 4800 -> 1200`, with mapped volume/uniform/tracer integral relative errors no larger than 2×10^{-10}. Fixed-decomposition MPI2 also passed.

Reversible-mode restriction: `undoableMeshCutter` ancestry is runtime state and is not serialized/distributed by this component. Start `automaticUnrefinement true` from the base/unrefined mesh. Restart from a written time containing active reversible splits, mesh-to-mesh mapping, and runtime mesh redistribution/load balancing are deliberately refused. Restart from a fully coarsened state was qualified and can refine again.

See the separate `planarRefiner_OF14_Manual` for complete theory, usage, and qualification details.

6.3 Native load balancing

Parallel dynamic load balancing uses OpenFOAM 14's native distributor:

```
distributor
{
    type                    loadBalancer;
    libs                    ("libfvMeshDistributors.so");
    redistributionInterval  5;
    maxImbalance            0;
    multiConstraint         false;
}
```

The redistribution method is taken from `system/decomposeParDict`. If Scotch redistribution is intended, the active dictionary during `foamRun` must contain:

```
method scotch;
```

The G4 qualification deliberately used a deterministic `simple` split for initial `decomposePar`, then switched the active dictionary to `scotch` before constructing the runtime load balancer.

6.4 Restart

In refinement-only mode, OpenFOAM owns the distributed mesh and registered field state. `planarRefiner` stores its global refinement-iteration counter in:

```
<time>/polyMesh/planarRefinerState
```

This prevents an already-refined restart from applying the same refinement iteration again. Refined/distributed restart was qualified against a continuous MPI trajectory.

With `automaticUnrefinement true`, the state dictionary additionally records whether reversible split history is active. Version 1.1.0 refuses a restart if active reversible splits were present at the write time, because reconstructing the `undoableMeshCutter` ancestry from the refined mesh alone is not safe. A restart from a fully coarsened state, where no reversible ancestry remains active, was qualified and can refine again.

7. Building and installing

7.1 Prerequisite

Source OpenFOAM Foundation 14 so that:

```
echo "$WM_PROJECT_VERSION"
```

returns `14`.

7.2 Full detonation solver build

From the release root:

```
./Allwmake
```

This builds:

```
$FOAM_USER_LIBBIN/libdetonationLegacyThermophysicalTransportModels.so
$FOAM_USER_LIBBIN/libdetonationFluidSolver.so
```

The optional 2-D/wedge AMR library is deliberately not required by this build.

7.3 AMR-only build

Build the reusable AMR component without detonationFoam:

```
./AllwmakeAMR
```

or:

```
cd src/planarFvMeshTopoChangers
./Allwmake
```

This builds only:

```
$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so
```

8. Running a case

8.1 Minimum `controlDict`

```
solver          detonationFluid;

libs
(
    "libdetonationLegacyThermophysicalTransportModels.so" // only when needed
);
```

For Sutherland/native transport, the compatibility library can be omitted if no compatibility Function2 or runtime registration is required by the case.

8.2 Solver-type selection

In `constant/solverTypeProperties`:

```
solverType              NS_Sutherland;
SW_position_limit       0.09999;
shockDetectionPressure  101425;
stopAtShockPosition     false;
```

Supported `solverType` selectors are `Euler` and `NS_Sutherland`. Exact legacy `NS_mixtureAverage` transport is selected through `physicalProperties` and `thermophysicalTransport`; it is not a third OF14 solver branch.

8.3 Flux selection

In `system/fvSchemes`:

```
fluxScheme HLLCP;
```

Choose one of `Kurganov`, `Tadmor`, `HLL`, `HLLC`, `HLLCP`, `AUSM+`, or `AUSM+up`.

8.4 Chemistry

A typical OpenFOAM 14 chemistry dictionary is:

```
type       standard;
chemistry  on;
cpuLoad    true;

initialChemicalTimeStep 1;

ode
{
    solver  seulex;
    absTol  1e-06;
    relTol  1e-03;
}
```

Mechanism data can be included from converted `foam/reactions.foam` and `foam/species.foam/thermo.foam` files.

8.5 Serial run

```
blockMesh
foamRun
```

The included fast tutorial can be run with:

```
cd tutorials/1D_NH3_O2_cracking_0.3_detonation_OF14_fast
./Allrun.serial
```

8.6 Parallel run

Prepare `system/decomposeParDict`, then:

```
decomposePar -force
mpirun -np 4 foamRun -parallel
```

For native runtime load balancing, add the `distributor` block to `constant/dynamicMeshDict` and ensure the active `decomposeParDict` specifies the desired runtime method, such as `scotch`.

8.7 Integrated H2/O2 laptop smoke

Version 1.1.0 includes a deliberately small 120-cell case that exercises chemistry, published-OF8 Soret compatibility, planar refinement, and automatic unrefinement together:

```
cd tutorials/H2_O2_laptop_autoUnref_Soret_OF14
./Allrun 2>&1 | tee log.laptopSmoke
```

The accepted qualification run produced repeated refinement and unrefinement events, reached `Tmax = 3581.900841 K`, advanced the leading shock to `0.00051 m`, and terminated normally without an OpenFOAM fatal error or floating-point exception.

The tutorial uses one explicitly **synthetic diagnostic** H2-O2 thermal-diffusion coefficient because the supplied OF8 archive did not include physical `thermoDiff` data. It demonstrates coupled software behavior; it is not a quantitative Soret or CJ-speed validation case.

9. Migrating an OpenFOAM 8 detonationFoam case to OpenFOAM 14

9.1 Migration workflow

A practical migration sequence is:

1. Copy the mesh and initial fields, preserving the physical initialization rather than re-creating the shock unless a new setup is intended.
2. Replace the standalone solver executable entry with `solver detonationFluid;` and run with `foamRun`.
3. Convert `thermophysicalProperties` to OpenFOAM 14 `physicalProperties` plus the standard multicomponent thermo data files.
4. Convert chemistry setup to OpenFOAM 14 `reactionProperties` and `chemistryProperties` using `type standard;`.
5. Move flux-family selection to `system/fvSchemes` with `fluxScheme`.
6. Retain `Euler` or `NS_Sutherland` in `constant/solverTypeProperties`. For legacy `NS_mixtureAverage`, configure the compatibility transport model rather than selecting a legacy solver branch.
7. Translate legacy binary-diffusion coefficients to the `legacyBinaryDiffusionCoefficient` Function2 entries when exact equivalence is required. The release includes `tools/ConvertLegacyBinaryDiff.py`.
8. Replace legacy/DLBFoam load balancing with the native OpenFOAM 14 `loadBalancer` and `cpuLoad true` chemistry accounting.
9. For 3-D AMR, use native OpenFOAM 14 `refiner`; for true 2-D or wedge AMR, optionally build and select `planarRefiner`. Enable `automaticUnrefinement` only when reversible coarsening is required and start from the base mesh.
10. If reproducing an OF8 case that used the published H/H2 Soret path, select `legacyThermalDiffusionMode publishedOF8` and provide the original explicit pair coefficients. Otherwise leave the default `off`.
11. Run a short fixed-time comparison before extending the simulation duration.

9.2 OF8 to OF14 capability disposition

OpenFOAM 8 capability	OpenFOAM 14 treatment (classification)	Notes
Standalone `detonationFoam` executable	`detonationFluid` selected by `foamRun` (Translated)	Modular OF14 solver architecture
Density-based shock solver structure	OF14 shock-fluid modular infrastructure (Native substitution)	Same density-based design intent with OF14 lifecycle and mesh hooks
`Kurganov` flux	Unified OF14 face-flux path (Retained)	Regression baseline
`Tadmor` flux	Unified OF14 face-flux path (Retained)	Qualified
`HLL`	Restored in `detonationFluid` (Translated)	Conservative OF14 implementation
`HLLC`	Restored in `detonationFluid` (Translated)	Contact-wave/star-state logic retained
`HLLCP`	Restored in	Pressure-corrected

OpenFOAM 8 capability	OpenFOAM 14 treatment (classification)	Notes
	`detonationFluid` (Translated)	detonation flux retained
`AUSM+`	Restored in `detonationFluid` (Translated)	Qualified
`AUSM+up`	Restored in `detonationFluid` (Translated)	Qualified
`Euler` solver type	`solverType Euler` (Translated/retained)	Inviscid transport path
`NS_Sutherland`	OF14 viscous momentum and thermophysical transport (Native substitution)	OF8/OF14 fixed-time profile comparison qualified
Separate `NS_mixtureAverage` equation files	Compatibility transport through OF14 `divj()`/`divq()` (Translated runtime model)	No duplicate solver branch
Legacy mixture-average D_i law	`legacyMixtureAverageFourier` (Translated exactly)	Uses $\left(1-Y_i\right)$ numerator
Legacy $D_{ij}\left(p,T\right)$ `Diff1..Diff4` law	`legacyBinaryDiffusionCoefficient` Function2 (Translated exactly)	Conversion tool supplied
Legacy log-polynomial species μ_i, λ_i	Native OF14 `logPolynomialTransport<8>` (Native substitution)	Converted coefficients evaluated directly
Legacy Wilke mixture viscosity	Native `coefficientWilkeMulticomponentMixture` (Native substitution)	Algebraic match verified to machine precision
Legacy arithmetic/harmonic mixture conductivity	`legacyKappa()` compatibility transport (Translated exactly)	Native Wilke conductivity is not equivalent
Species sensible-enthalpy diffusion	OF14 multicomponent `divq()` plus translated closure	Uses corrected legacy species flux
DLBFoam/load-balanced chemistry	Standard OF14 chemistry, `cpuLoad true`, native `loadBalancer` (Native substitution)	Old DLBFoam dependency intentionally removed
Legacy/custom parallel redistribution	Native `fvMeshDistributors::loadBalancer` with Scotch (Native substitution)	MPI/restart qualified
Legacy/custom 3-D AMR dependency	Native `fvMeshTopoChangers::refiner` (Native substitution)	Serial/MPI/restart qualified
True 2-D/axisymmetric AMR	Reusable `planarRefiner` library (New OF14 component)	Independent of detonationFoam
Published legacy Soret H/H2 behavior	`publishedOF8` mode (Explicit legacy reproduction)	Reproduces the OF8 H2-to-`TDRatio_H` assignment; default is `off`
Automatic unrefinement in	v1.1.0 reversible coarsening	Qualified for slab, localized

OpenFOAM 8 capability	OpenFOAM 14 treatment (classification)	Notes
`planarRefiner`	(New OF14 extension)	slab, wedge, and fixed-decomposition MPI2; active-history restart and runtime redistribution are blocked
Full-resolution long OF8/OF14 equivalence and formal grid convergence	Deferred post-release	Laptop release gates are intentionally short
Formal strong/weak scalability study	Deferred post-release	G4 is a compact infrastructure/timing smoke only

9.3 Important migration detail for `NS_mixtureAverage`

Do not write:

```
solverType NS_mixtureAverage;
```

in the OF14 case. The release intentionally avoids reviving that legacy equation branch. Use the viscous solver path and select the compatibility physics through the thermo/transport dictionaries, for example:

```
// constant/physicalProperties
thermoType
{
    type            hePsiThermo;
    mixture         coefficientWilkeMulticomponentMixture;
    transport       logPolynomial;
    thermo          janaf;
    energy          sensibleInternalEnergy;
    equationOfState perfectGas;
    specie          specie;
}
```

and:

```
// constant/thermophysicalTransport
laminar
{
    model legacyMixtureAverageFourier;
    legacyThermalDiffusionMode off;
    D
    {
        // one legacyBinaryDiffusionCoefficient entry per binary pair
    }
}
```

10. Qualification summary

The v1.0.0 migration closed Stages A-G and the clean R1 release gate. Version 1.1.0 retains that evidence and adds three feature-closure gates.

Qualification area	Main evidence
A-D	modular `foamRun` port, seven flux families, serial/MPI/restart matrix, exact legacy transport/property compatibility
E-F	native OF14 3-D AMR plus reusable true-2-D/wedge `planarRefiner`
G1	integrated laptop release regression - 205 s

Qualification area	Main evidence
G2	compact OF8/OF14 equivalence replay - 31 s
G3	mesh/time-step characterization - 249 s
G4	serial/MPI2/MPI4 native load-balancer smoke - 348 s
R1	clean package/build/install/runtime smoke
v1.1-U	automatic planar unrefinement runtime gate - **PASS**
v1.1-S	published-OF8 H/H2 Soret flux gate - **PASS**
v1.1-I	integrated 120-cell H2/O2 laptop smoke - **PASS**

v1.1-U automatic unrefinement

Accepted reversible cycles were:

- uniform slab: `1200 -> 4800 -> 1200 -> 4800 -> 1200`;
- localized slab: `1200 -> 2352 -> 1200 -> 2352 -> 1200`;
- annular wedge: `1600 -> 6400 -> 1600 -> 6400 -> 1600`.

Mapped volume, uniform-field, and tracer integrals were conserved to relative tolerance $\leq 2 \times 10^{-10}$. Serial `checkMesh` passed at the topology states, fixed-decomposition MPI2 passed, active-history restart was correctly refused, and restart from a fully coarsened state followed by re-refinement passed.

v1.1-S published OF8 H/H2 Soret

The direct species-flux probe passed with a synthetic H2-only diagnostic polynomial:

Diagnostic	Result
Maximum absolute H flux in `publishedOF8` mode	`1.690421e-03`
inferred direct H2 Soret residual	`1.595e-16` relative
published H/H2 corrected-flux identity	`1.603e-16` relative
zero-net-mass species-flux closure	`1.283e-16` relative
Soret-off H/H2 flux	exactly zero

v1.1-I integrated H2/O2 smoke

The 120-cell coupled smoke constructed both new modes, produced repeated refinement and automatic-unrefinement events, reached a final temperature range of `300 .. 3581.900841 K`, advanced the leading shock to `0.00051 m`, and terminated normally without an OpenFOAM fatal error/FPE.

The smoke uses a synthetic thermal-diffusion coefficient and therefore verifies integration, not physical Soret accuracy.

Selected retained G3 sensitivity results at $t = 2.7 \times 10^{-8}$ s are:

Mesh/time-step case	Shock [m]	p_{max} [Pa]	T_{max} [K]
10 um, Co 0.10	0.0102634382	1,986,615.73	4994.4747
5 um, Co 0.10	0.0102627687	2,171,901.06	4994.47564
2.5 um, Co 0.10	0.0102627994	2,262,087.22	4994.47605
5 um, Co 0.05	0.0102627957	2,152,715.98	4994.47583

The shock position was already very stable from 5 to 2.5 um, while peak pressure remained more resolution-sensitive. This remains screening characterization rather than a claim of asymptotic grid convergence.

11. Known limitations and post-release work

The following items are intentionally not release-blocking:

- full-resolution, long-duration OF8-to-OF14 equivalence runs;
- formal grid/time-step convergence and CJ-speed/experimental validation;
- long endurance runs with repeated AMR operations;
- formal strong/weak parallel scalability studies;
- a separately derived and validated **corrected** H/H2 Soret formulation, distinct from `publishedOF8`;
- validated physical H/H2 thermal-diffusion coefficients for target mechanisms where they are not already available;
- serialization/distribution of reversible `planarRefiner` ancestry so automatic unrefinement can support active-history restart and runtime mesh redistribution/load balancing.

The release qualification policy targets approximately 30 minutes or less per gate on a laptop. Longer studies are documented in `qualification/POST_RELEASE_DEFERRED_QUALIFICATION.md`.

12. Troubleshooting

12.1 `foamRun` cannot select `detonationFluid`

Confirm the full build completed and that:

```
ls "$FOAM_USER_LIBBIN/libdetonationFluidSolver.so"
```

exists. Also confirm `system/controlDict` contains `solver detonationFluid;`.

12.2 `legacyMixtureAverageFourier` is not found

Ensure `libdetonationLegacyThermophysicalTransportModels.so` was built and loaded in `controlDict`:

```
libs
(
    "libdetonationLegacyThermophysicalTransportModels.so"
);
```

12.3 AMR library is not found

Build it independently:

```
./AllwmakeAMR
```

and confirm:

```
ls "$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so"
```

12.4 Load balancer says `Selecting distributor simple`

The native load balancer reads the active `system/decomposeParDict`. If runtime Scotch redistribution is intended, switch the active dictionary to:

```
method scotch;
```

before `foamRun` starts.

12.5 Localized 2-D AMR creates a few non-hex transition cells

This is expected for the qualified directional local-refinement topology. The reference slab case retained bounded transition polyhedra with low non-orthogonality/skewness and good species closure. Uniform planar refinement and the wedge portability case remained strict `Mesh OK.` cases.

12.6 Automatic unrefinement does not trigger

Check that:

- `automaticUnrefinement true;` is present;
- the current time index satisfies `unrefineInterval`;
- reversible splits were created during the same uninterrupted run;
- both siblings of a candidate split lie below `lowerUnrefineLevel` or both lie above `upperUnrefineLevel`;
- the hysteresis thresholds satisfy `lowerUnrefineLevel <= lowerRefineLevel` and `upperUnrefineLevel >= upperRefineLevel`.

A split that straddles the indicator band is intentionally retained.

12.7 Reversible-mode restart or load balancing is refused

This is an intentional v1.1.0 safety restriction, not a crash. Active `undoableMeshCutter` ancestry is not serialized or redistributed. Start reversible mode from the base mesh, use fixed decomposition while reversible history is active, or restart only after the mesh has fully coarsened.

12.8 `publishedOF8` reports a missing thermal-diffusion pair

The v1.1.0 compatibility mode requires explicit H/H2 pair dictionaries. Supply the missing pair in `constant/thermoDiff` or inline under `thermalDiffusionCoeffs`. The old `trandat/groupSpecies` coefficient-sharing shortcut is intentionally not inferred.

12.9 The H2 direct Soret term is zero

That is expected in `legacyThermalDiffusionMode publishedOF8`. The mode intentionally reproduces the supplied OF8 source assignment in which the H2 polynomial sum is accumulated into the H ratio. Use `off` when that legacy behavior is not desired; a separately validated corrected H/H2 formulation is outside the scope of v1.1.0.

13. File map

```
applications/modules/detonationFluid/
    OpenFOAM 14 foamRun detonation solver
src/detonationLegacyThermophysicalTransportModels/
    exact legacy mixture-average/property compatibility library
src/planarFvMeshTopoChangers/
    reusable 2-D slab/wedge AMR library
tutorials/
    migrated NH3/O2 examples, OF8 comparison reference, and 120-cell H2/O2 v1.1 integration
smoke
tools/
    legacy binary-diffusion conversion support
docs/
    this manual and standalone planarRefiner manual in Markdown/Word/PDF
qualification/
    compact gate evidence, v1.1 automatic-unrefinement/Soret closure, and deferred post-
release notes
```